```
        }
        else
        {
            Iterator const mid_point=first+(length/2);     <-7
            std::future<Iterator> async_result=
                std::async(&parallel_find_impl<Iterator,MatchType>,     <-8
                           mid_point,last,match,std::ref(done));
            Iterator const direct_result=
                    parallel_find_impl(first,mid_point,match,done);     <-9
            return (direct_result==mid_point)?
                async_result.get():direct_result;     <-10
        }
    }
    catch(...)
    {
        done=true;     <-11
        throw;
    }
}

template<typename Iterator,typename MatchType>
Iterator parallel_find(Iterator first,Iterator last,MatchType match)
{
    std::atomic<bool> done(false);
    return parallel_find_impl(first,last,match,done);     <-12
}
```

The desire to finish early if you find a match means that you need to introduce a flag that is shared between all threads to indicate that a match has been found. This therefore needs to be passed in to all recursive calls. The simplest way to achieve this is by delegating to an implementation function ❶ that takes an additional parameter—a reference to the `done` flag, which is passed in from the main entry point ⓬.

The core implementation then proceeds along familiar lines. In common with many of the implementations here, you set a minimum number of items to process on a single thread ❷; if you can't cleanly divide into two halves of at least that size, you run everything on the current thread ❸. The actual algorithm is a simple loop through the specified range, looping until you reach the end of the range or the `done` flag is set ❹. If you do find a match, the `done` flag is set before returning ❺. If you stop searching either because you got to the end of the list or because another thread set the `done` flag, you return `last` to indicate that no match was found here ❻.